

Моё руководство хочет чтобы я провел UI нагрузочное тестирование, именно нагрузочное тестирование пользовательского интерфейса.

Например:

100 пользователей в течении 1 минуты должны открыть страницу сайта, нажать кнопку "войти", заполнить поля формы (логин и пароль) и нажать кнопку "авторизоваться" в результате проверить поп-ап с сообщением "Вы успешно авторизованы".

Скажите пожалуйста каким инструментом можно выполнить такое нагрузочное тестирование ?

 Thought for 3 seconds >

Для нагрузочного тестирования пользовательского интерфейса (UI) с описанным вами сценарием (открытие страницы, заполнение формы, проверка всплывающего сообщения) можно использовать несколько подходов и инструментов.

Главное различие между ними — имитируют ли они действия на уровне HTTP-запросов или полноценно работают с браузером (как реальный пользователь).

Основные подходы и инструменты

1. Комбинация: Нагрузка (API/HTTP) + Функциональная проверка (UI)

Это самый практичный и масштабируемый подход для вашей задачи. Основную нагрузку создают легковесные HTTP-запросы, а для проверки UI (появления поп-апа) используется один или несколько браузерных экземпляров.

- Как это работает:
 - **Инструмент нагрузки** (например, **JMeter** или **k6**) отправляет тысячи HTTP-запросов, имитируя логин-процесс (POST /login с данными формы) для создания пиковой нагрузки ☞ .
 - **Инструмент UI-автоматизации** (например, **Selenium** или **Playwright**) одновременно выполняет ваш сценарий в реальном браузере в несколько потоков (например, 5-10 сессий), проверяя появление поп-апа ☞ .
- Инструменты:

- **Apache JMeter + Selenium WebDriver:** Классическое и очень популярное открытое решение [↗](#) . JMeter создает нагрузку, а Selenium проверяет функциональность [↗](#) .
- **k6 + Playwright:** Современный подход. k6 отлично справляется с нагрузкой [↗](#) , а Playwright — мощный и быстрый инструмент для работы с браузером [↗](#) . k6 также имеет собственный модуль для браузерных тестов [↗](#) .

2. Полноценное браузерное тестирование (Browser-based)

Этот подход максимально реалистичен, так как каждый виртуальный пользователь использует настоящий браузер. Однако он требует огромных ресурсов и плохо масштабируется для 100 пользователей.

- **Как это работает:** Инструмент управляет реальными браузерами (или их "движками") для каждого виртуального пользователя, выполняя все клики, ввод данных и проверки.
- **Инструменты:**
 - **Playwright и Selenium:** Хотя они созданы для функционального тестирования, их можно использовать в нагрузочных сценариях, запуская

параллельно [↗](#) . Однако для 100 одновременных пользователей это может быть крайне ресурсозатратно.

- **LoadNinja:** Облачная платформа, которая позволяет создавать и запускать нагрузочные тесты прямо в браузере [↗](#) .



Сравнение некоторых инструментов

Инструмент	Тип	Ключевые особенности
Apache JMeter	Нагрузка (HTTP/API)	Бесплатный, очень популярный, есть GUI для создания сценариев, мощное сообщество ↗ .
k6	Нагрузка (HTTP/API)	Современный, код-ориентированный (на JavaScript), отлично масштабируется, разработан Grafana Labs ↗ .
Locust	Нагрузка (HTTP/API)	Основан на Python, сценарии пишутся как код, имеет удобный

Инструмент	Тип	Ключевые особенности
		веб-интерфейс для управления тестом в реальном времени 🔗 .
Gatling	Нагрузка (HTTP/API)	Мощный инструмент для Java/Scala-команд, отличная производительность 🔗 .
Selenium	Браузерная автоматизация	Самый известный инструмент для автоматизации действий в браузере 🔗 .
Playwright	Браузерная автоматизация	Современная, быстрая и надежная альтернатива Selenium от Microsoft 🔗 .
BlazeMeter	Облачная платформа	Позволяет запускать JMeter-тесты в облаке, легко масштабируется 🔗 .
LoadNinja	Облачная платформа	Специализируется на браузерном нагрузочном тестировании без



Рекомендация для вашего сценария

Для вашей конкретной задачи ("100 пользователей за 1 минуту") я бы рекомендовал **комбинированный подход**:

1. **Создайте нагрузку с помощью JMeter или k6** . Эти инструменты легко создадут 100 параллельных сессий, отправляя HTTP-запросы на вход. Это эффективно и не требует огромных ресурсов.
2. **Для проверки UI используйте Selenium или Playwright** . Запустите небольшое количество (например, 2-3) браузерных сессий, которые будут выполнять тот же сценарий входа и проверять появление поп-апа. Это подтвердит, что система не только выдерживает нагрузку, но и корректно отображает интерфейс для пользователя.

Этот подход даст вам и надежные цифры по производительности, и уверенность в корректной работе пользовательского интерфейса под нагрузкой.

Если у вас есть вопросы по настройке какого-либо из этих инструментов, спрашивайте.



46 web pages